Because of the compact array implementation there will be no pointers to redirect, and structure will be guaranteed to be consistent with properties 1 (by construction) and 2 (unless you have bugs in your code, of course).

So, we have to guarantee property 3, i.e. that each node is smaller than each one of its `D` children. To do so, as we mentioned in previous sections, we might have to push it down (on deletes, and possibly updates), or bubble it up towards the root (on insert, and updates as well), whenever heap invariants are violated.

How many swaps will we need to perform in order to reinstate the heap properties? That depends on the concrete case, but since we move one level up at each swap, it can't be more than the heap's height. And since a heap is a balanced complete tree, that means no more than $\log_D(n)$ swaps, for a `D`-way heap with `n` elements.

The caveat here is that in any concrete implementation we need to expand the array beyond its current size. If we use a fixed-size static array, we will need to allocate it when the heap is created and we need to set the max number of elements it can hold. In this case, however, the method is logarithmic in the worst case.

If we use a dynamic array, then the logarithmic bound becomes amortized and not worst-case, because we will periodically need to resize the array (see appendix C to check out more on dynamic arrays performance).
